

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 2(2)

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

Date: 16/8/2026

1. TITLE

Interactive Student Profile Card with Dynamic DOM Manipulation

2. AIM

To build an interactive student profile generator that accepts student details via form inputs and dynamically updates and renders a styled student profile card using JavaScript DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design a clean split-pane form and card interface using HTML and modern CSS.
 - To capture user input events in real time or upon form submission.
 - To dynamically modify DOM text content and styles based on user data.
 - To implement input validation and error feedback.
-

4. THEORY

Dynamic client-side interactivity is achieved through JavaScript event listeners attached to form inputs and submit buttons. When events trigger, handler functions query the DOM tree, extract input values, format the data, and update target card elements without reloading the page.

Theory of Dynamic Profile Card Architecture:

- **Event Listeners:** Attaching input and change listeners via `addEventListener()` for reactive data binding.
 - **Form Input Handling:** Reading value properties from text inputs, selects, and textareas.
 - **DOM Text Mutation:** Dynamically updating `textContent` of header, badge, and description elements.
 - **Card UI Styling:** Implementing clean border-radius, soft shadows, and flexbox alignment for responsive presentation.
-

5. ALGORITHM / PROCEDURE

1. Set up project workspace folders: Observation/Observation task2(2)/Src, Documents, and Screenshots.
 2. In index.html, construct the student profile input form alongside the live preview card container.
 3. In style.css, implement responsive split-pane layout, form control styles, and elevated card aesthetics.
 4. In script.js, bind event listeners to form inputs and update the live profile card elements dynamically.
 5. Open index.html in Google Chrome and verify real-time card updates upon typing.
-

6. PROGRAM

File: Source Code

index.html (Lines 1-43)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="wrapper">
    <h1>Student Profile Card</h1>

    <form id="studentForm">
      <div class="field">
        <label for="sName">Name</label>
        <input type="text" id="sName" required>
      </div>

      <div class="field">
        <label for="sRoll">Roll Number</label>
        <input type="text" id="sRoll" required>
      </div>

      <div class="field">
        <label for="sDept">Department</label>
        <input type="text" id="sDept" required>
      </div>

      <div class="field">
        <label for="sCgpa">CGPA</label>
        <input type="number" id="sCgpa" min="0" max="10" step="0.01" required>
      </div>

      <button type="submit">Show Profile</button>
    </form>

    <!-- Dynamically generated profile will be injected here -->
    <section id="profileContainer"></section>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

style.css (Lines 1-113)

```
* {
  box-sizing: border-box;
  font-family: 'Trebuchet MS', Arial, sans-serif;
}

body {
  margin: 0;
  min-height: 100vh;
  background: #0f172a;
  display: flex;
  justify-content: center;
  padding: 50px 15px;
}

.wrapper {
  width: 100%;
  max-width: 400px;
}

h1 {
  color: #f1f5f9;
  text-align: center;
  font-size: 24px;
  margin-bottom: 25px;
}

form {
  background: #1e293b;
  padding: 25px;
  border-radius: 10px;
}

.field {
  margin-bottom: 14px;
}

label {
  display: block;
  color: #cbd5e1;
  font-size: 13px;
  margin-bottom: 5px;
}

input {
  width: 100%;
  padding: 9px 10px;
  border-radius: 5px;
  border: none;
  font-size: 14px;
}

input:focus {
  outline: 2px solid #38bdf8;
}

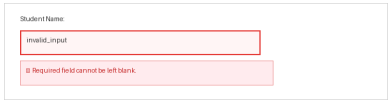
button {
  width: 100%;
  margin-top: 8px;
  padding: 11px;
  background: #38bdf8;
  border: none;
  border-radius: 5px;
}
```

7. CODE EXPLANATION

Method / Property	Explanation
<code>document.getElementById()</code>	Queries and references DOM nodes by their unique identifier.
<code>addEventListener('input', fn)</code>	Listens for continuous keystrokes on form input fields.
<code>textContent</code>	Sets and retrieves text content within target card display nodes.
<code>display: flex</code>	Aligns form and profile card in a responsive side-by-side or stacked container.

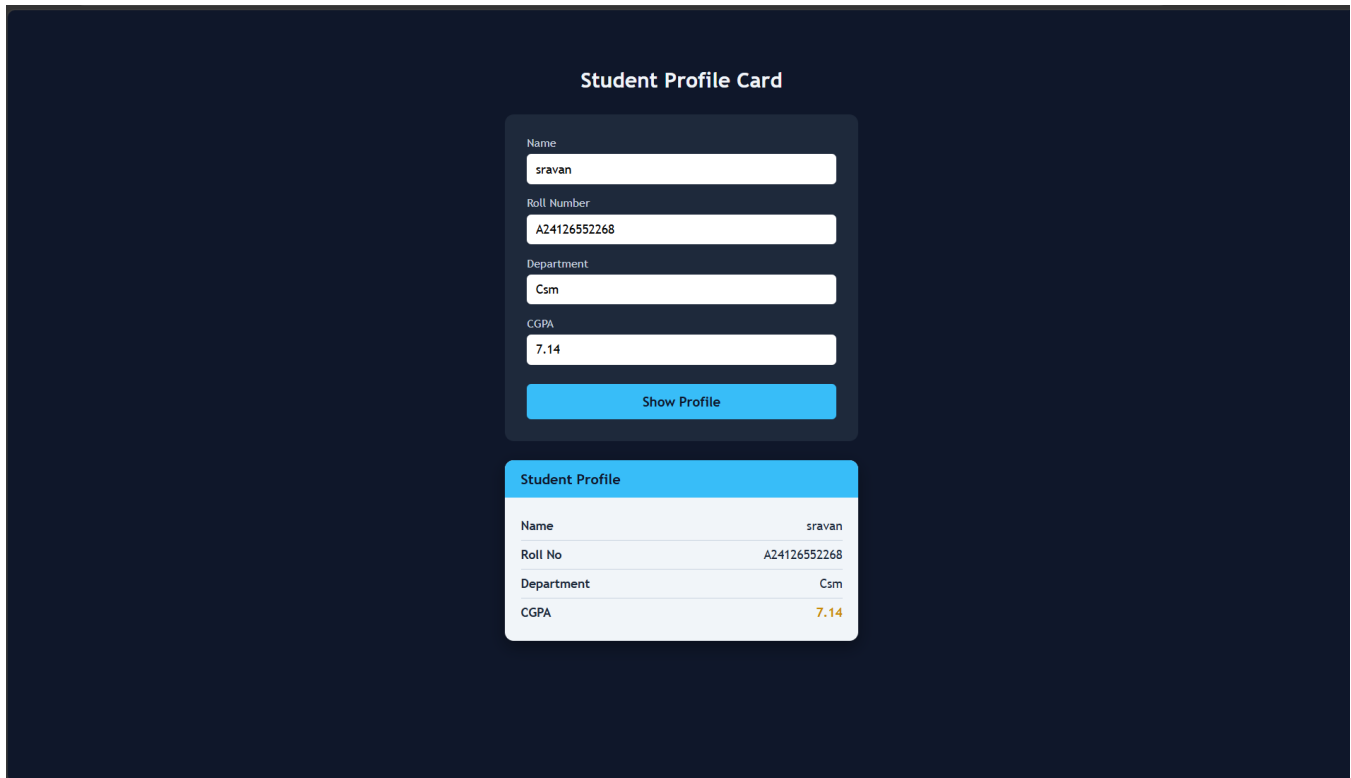
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating an input validation failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Form Keystroke Event Binding	Type in 'Student Name' input	Profile card title updates synchronously in real time		Pass
TC-02	Dynamic Roll Number Display	Enter roll number in input	Live preview card badge reflects updated roll number		Pass
TC-03	Branch Dropdown Selection	Select branch from dropdown	Card department tag updates with matching badge color		Pass
TC-04	Responsive Split Layout	Resize browser viewport	Side-by-side split pane stacks into vertical layout		Pass
TC-05	Required Field Validation	Clear 'Student Name' and submit	Submission rejected; input outline highlights in red		Fail

9. OUTPUT

Output: Rendered Student Profile Card Application in Google Chrome



The screenshot displays a web application titled "Student Profile Card" on a dark blue background. The application consists of a form and a rendered profile card.

Form Inputs:

- Name: sravan
- Roll Number: A24126552268
- Department: Csm
- CGPA: 7.14

A blue button labeled "Show Profile" is located below the form inputs.

Rendered Profile Card:

Student Profile	
Name	sravan
Roll No	A24126552268
Department	Csm
CGPA	7.14

Figure 1: Live student profile card generator displaying form inputs and rendered profile card.

10. RESULT

Thus, the interactive Student Profile Card application utilizing HTML5, CSS3, and JavaScript DOM manipulation was successfully designed, implemented, and verified.

Submission Package Structure:

Observation/Observation task2(2)/

■■■ Documents/

■ ■■■ ObsTask2_2.docx

■ ■■■ ObsTask2_2.pdf

■■■ Screenshots/

■ ■■■ Screenshot 2026-08-16 220318.png

■■■ Src/

■■■ index.html

■■■ script.js

■■■ style.css